

Auto-refinamiento y ReAct

Módulo 6 · Ingeniería de Prompts

Ciclo critica-mejora, ReAct Thought/Action/Observation, function calling y el log de agentes como herramienta de debugging.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

Auto-refinamiento y ReAct

El auto-refinamiento y ReAct son dos técnicas que introducen feedback loops en el proceso de generación de LLMs. El auto-refinamiento toma la respuesta inicial del modelo y la usa como input para una segunda llamada que la critica y mejora. ReAct intercala razonamiento con acciones (llamadas a herramientas externas) para que el modelo pueda verificar su razonamiento contra fuentes externas. Ambas técnicas abordan la misma limitación fundamental: un LLM que genera en una sola pasada no tiene mecanismo para detectar y corregir sus propios errores.

En 2023, la adopción masiva de agentes LLM en producción (LangChain, LlamaIndex, AutoGPT) popularizó estos patrones. En 2025, ReAct es la arquitectura de referencia para sistemas agénticos que necesitan combinar razonamiento con acceso a información externa. El auto-refinamiento es una práctica estándar en sistemas de generación de contenido y de código donde la calidad del output importa más que la velocidad de respuesta.

Auto-refinamiento: el modelo critica y mejora su propio output

El ciclo básico de auto-refinamiento

Auto-refinamiento (Madaan et al., 2023) sigue tres pasos: generar una respuesta inicial (con temperatura moderada para producir un buen primer borrador), generar retroalimentación (usando un prompt específico que pide al modelo que critique su propia respuesta, identificando errores, omisiones y formas de mejorarla), y refinar (usando la respuesta inicial y la retroalimentación para generar una versión mejorada). El ciclo puede repetirse 1-3 veces.

El prompt de retroalimentación es tan importante como el prompt de generación. Un buen prompt de retroalimentación especifica exactamente qué criterios evaluar: "Revisa la respuesta anterior. Evalúa: (1) precisión factual, (2) completitud (¿falta algún aspecto importante?), (3) claridad y estructura, (4) adecuación al público objetivo. Para cada criterio, identifica deficiencias específicas y sugiere mejoras concretas."

ReAct: razonar y actuar en bucles interleaved

ReAct (Yao et al., 2023, arXiv:2210.03629) propone un paradigma donde el LLM alterna entre pensamientos (razonamiento interno) y acciones (llamadas a herramientas externas como búsqueda web, calculadoras, APIs) en un bucle iterativo. El ciclo es: Thought (razonar sobre el estado actual y qué hacer), Action (ejecutar una herramienta o acción), Observation (procesar el resultado de la acción), y volver a Thought con el nuevo contexto. Este patrón permite al modelo verificar su razonamiento contra fuentes externas y ajustar su plan basándose en los resultados reales de las acciones.

Implementación práctica de ReAct con function calling

Con la API de OpenAI, la implementación de ReAct usa function calling: defines las herramientas disponibles (funciones Python con nombre, descripción y schema de parámetros), envías el mensaje al modelo con la lista de herramientas, si el modelo decide usar una herramienta el API devuelve un `tool_use` message con el nombre de la función y los argumentos, ejecutas la función localmente y devuelves el resultado como `tool_result`, y continúas el diálogo hasta que el modelo produce un mensaje final sin `tool_use`.

El bucle de ReAct completo en código es un while loop: mientras el modelo responda con `tool_use`, ejecuta la herramienta y añade el resultado al historial de mensajes. El número máximo de iteraciones debe limitarse para evitar bucles infinitos (típicamente 10-25 pasos como máximo). Cada iteración añade tokens al contexto acumulado, por lo que el coste total puede ser significativamente mayor que una sola llamada.

Reflexion: auto-refinamiento con memoria episódica

Reflexion (Shinn et al., 2023) extiende ReAct añadiendo una memoria episódica: el agente no solo actúa y observa, sino que también genera reflexiones verbales sobre lo que aprendió de cada intento. Estas reflexiones se almacenan y se incluyen en el contexto de los siguientes intentos, permitiendo al modelo evitar cometer los mismos errores. Es especialmente efectivo en tareas iterativas donde el modelo necesita aprender de los fallos de intentos anteriores.

SECCIÓN 4 · EJEMPLOS REALES

- Auto-refinamiento en redacción de correos: primera pasada → borrador del correo. Retroalimentación → identificar que el tono es demasiado formal para el destinatario y que falta una llamada a la acción clara. Segunda pasada → correo mejorado con tono más cercano y CTA explícita. Este ciclo de dos pasos mejora la calidad del correo mejor que una única instrucción detallada.
- ReAct para investigación automatizada: un agente de investigación usa ReAct con herramientas de búsqueda web y lectura de PDF. Thought: "Necesito datos sobre el mercado de energía solar en España". Action: search("mercado energía solar España 2024"). Observation: [resultados]. Thought: "Los datos de instalaciones los tiene el IDAE". Action: fetch_url("https://idaes.es/..."). Observation: [datos]. Thought: "Ahora tengo suficiente para el análisis." Response: [análisis completo].
- Auto-refinamiento de código: primer borrador del código → retroalimentación automatizada (ejecutar linters, ejecutar tests, analizar complejidad ciclomática) → segunda pasada con las correcciones. GitHub Copilot con auto-edit usa un ciclo similar: genera código, ejecuta los tests, si fallan propone correcciones en base a los mensajes de error.
- Reflexion en sistemas de ticketing: un agente de resolución de tickets de soporte técnico registra en memoria los errores que cometió en tickets similares anteriores ("en el ticket #1234 intenté resetear la contraseña cuando el problema era el MFA, lo que retrasó la resolución 2 horas"). En tickets futuros similares, consulta esta memoria y evita el mismo error.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: Asumir que el auto-refinamiento siempre mejora el output. Si el modelo está mal calibrado para la tarea de retroalimentación, puede producir críticas incorrectas y hacer el output peor en la segunda pasada. Evalúa el ciclo completo con tu golden dataset para verificar que la segunda pasada realmente mejora sobre la primera.

Error 2: No limitar las iteraciones de ReAct. Sin un límite máximo de iteraciones, un agente puede entrar en bucles que consumen cientos de llamadas al API. Define siempre `max_iterations` y gestiona el caso donde el agente alcanza el límite sin haber completado la tarea.

Error 3: Confiar en ReAct para tareas que requieren conocimiento interno preciso. ReAct es excelente para recuperar información externa, pero si el razonamiento en los Thought steps es incorrecto, las acciones serán incorrectas independientemente de los resultados que devuelvan las herramientas. ReAct no corrige errores de razonamiento, solo añade acceso a información externa.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

Para implementar auto-refinamiento: diseña un prompt de retroalimentación específico para tu caso de uso (¿qué criterios de calidad importan?), limita el ciclo a 2-3 iteraciones (el retorno marginal decrece rápidamente), evalúa si la respuesta de la segunda pasada es realmente mejor que la primera sobre tu golden dataset (no siempre lo es), y decide si el coste adicional en tokens y latencia está justificado por la mejora de calidad.

Para implementar ReAct con function calling: define herramientas con descripciones precisas (el modelo decide qué herramienta usar basándose en la descripción), implementa el bucle de ejecución con límite de iteraciones y gestión de errores de herramientas, añade logging de cada Thought/Action/Observation para debugging, y define cuándo el agente debe parar (`task_complete` flag o N iteraciones máximas).

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M10 (Agentes): ReAct es la arquitectura base de la mayoría de los agentes LLM modernos.
- M6-T2 (CoT): ReAct extiende CoT añadiendo la capacidad de tomar acciones basadas en el razonamiento.
- M8 (LLMOps): el logging de Thought/Action/Observation de ReAct es el componente de observabilidad más importante de los sistemas agénticos.

SECCIÓN 8 · RESUMEN EJECUTIVO

El auto-refinamiento usa el modelo para criticar y mejorar su propio output: generar borrador → generar retroalimentación estructurada → refinar. Mejora la calidad cuando el prompt de retroalimentación está bien diseñado, pero no siempre garantiza mejora; evalúa con golden dataset. ReAct (arXiv:2210.03629) interleva razonamiento (Thought) con acciones (Action/Observation), permitiendo al modelo verificar su razonamiento contra fuentes externas. Implementado con function calling en GPT-4/Claude/Gemini. Siempre limita el número máximo de iteraciones. El log de Thought/Action/Observation es imprescindible para debugging. Reflexion extiende ReAct con memoria episódica de errores pasados.